

BÁO CÁO ĐỒ ÁN MÔN HỌC XÂY DỰNG THƯ VIỆN SỬ DỤNG TEMPLATES

Học phần: Cấu trúc dữ liệu và giải thuật

Lớp: đã xóa

Thực hiện bởi:

Lý Nguyễn Công Chính

Kiều Gia Khang

I. QUÁ TRÌNH CÀI ĐẶT THƯ VIỆN VÀ ỨNG DỤNG

1. Lý Nguyễn Công Chính – Xây dựng thư viện algoracoon

Chính tập trung xây dựng thư viện C++ tổng quát sử dụng cơ chế Template, đặt trong namespace algoracoon. Để tránh lỗi liên kết khi phân giải template tại thời điểm biên dịch, toàn bộ mã nguồn của mỗi cấu trúc được định nghĩa trực tiếp trong các tệp tiêu đề .hpp tương ứng.

Các cấu trúc dữ liệu và giải thuật đã được cài đặt bao gồm:

- **LinkedList.hpp (Danh sách liên kết đơn):** Cài đặt danh sách liên kết đơn tổng quát hỗ trợ thêm, xóa, tìm kiếm, lưu kích thước và dọn dẹp bộ nhớ. Hỗ trợ duyệt danh sách theo cả hai chiều từ đầu về cuối và ngược lại.
- **Stack.hpp (Ngăn xếp):** Cài đặt cấu trúc ngăn xếp LIFO sử dụng mảng động tự co giãn.
- **Queue.hpp (Hàng đợi):** Cài đặt cấu trúc hàng đợi FIFO sử dụng bộ đệm vòng (circular buffer) trên mảng động.
- **PriorityQueue.hpp (Hàng đợi ưu tiên):** Cài đặt hàng đợi ưu tiên sử dụng cấu trúc Max-Heap làm mặc định, hỗ trợ thay đổi bộ so sánh.
- **Algorithms.hpp (Sắp xếp và Tìm kiếm):** Cung cấp các thuật toán sắp xếp (Bubble, Selection, Insertion, Heap, Quick, Merge) nhận bộ so sánh tùy chọn và các giải thuật tìm kiếm (Tuyến tính, Nhị phân).
- **BST.hpp (Cây tìm kiếm nhị phân):** Cây tìm kiếm nhị phân chuẩn hỗ trợ thêm, xóa, tìm kiếm và duyệt cây theo 3 thứ tự tiền/trung/hậu.
- **AVL.hpp (Cây tự cân bằng AVL):** Mở rộng từ BST với cơ chế tự động xoay cân bằng trái/phải sau mỗi thao tác chèn/xóa để đảm bảo chiều cao tối ưu $O(\log n)$.
- **HashTable.hpp (Bảng băm chuỗi AVL):** Cài đặt bảng băm giải quyết độ bằng phương pháp Separate Chaining, trong đó mỗi bucket là một cây AVL thay vì danh sách liên kết thông thường nhằm giảm thiểu thời gian tra cứu khi độ lớn.

2. Kiều Gia Khang - Phát triển ứng dụng giả lập Cache (Cache Simulator)

Khang xây dựng ứng dụng mô phỏng cơ chế quản lý bộ nhớ đệm Cache sử dụng giao diện dòng lệnh (CLI). Ứng dụng tích hợp sâu sắc 5 trên 7 cấu trúc dữ liệu của thư viện để thực hiện các chức năng thực tế:

- **BST**: Quản lý bảng ánh xạ từ địa chỉ ảo sang địa chỉ vật lý.
- **AVL**: Mô phỏng bộ nhớ chính lưu trữ dữ liệu thực tế.
- **HashTable**: Đóng vai trò là bộ nhớ đệm Cache lưu trữ các khối nhớ đang được tải lên.
- **Queue**: Quản lý hàng đợi các yêu cầu truy cập bộ nhớ gửi từ CPU.
- **LinkedList**: Quản lý thứ tự giải phóng các khối nhớ trong Cache theo chiến lược LRU (Least Recently Used).
- **Stack**: Lưu nhật ký (log) các hoạt động gần nhất của hệ thống.
- **Bubble Sort**: Thống kê lượt truy cập của các khối nhớ và sắp xếp báo cáo theo tần suất giảm dần.
- **Priority Queue**: Quản lý độ ưu tiên giải phóng bộ nhớ của các khối nhớ theo chiến lược LFU (Least Frequently Used).

II. BÁO CÁO CÁC TEST CASE VÀ KẾT QUẢ CHẠY THỰC TẾ

1. Kiểm thử chương trình kiểm thử thư viện (tests/main.cpp)

Chương trình kiểm thử thư viện được thiết kế để tự động kiểm tra hoạt động của từng cấu trúc dữ liệu đơn lẻ trong thư mục lib/. Các test case cụ thể bao gồm:

Cấu trúc dữ liệu	Kịch bản kiểm thử (Test Case)	Kết quả mong đợi
LinkedList	Chèn đầu (1), chèn cuối (2), chèn vị trí (3). Duyệt xuôi và ngược.	Duyệt xuôi: 1 2 3 Duyệt ngược: 3 2 1
Stack	Đẩy 10 và 20 vào Stack, lấy phần tử đỉnh.	Stack top: 20
Queue	Xếp hàng 'first' và 'second', lấy phần tử đầu hàng.	Queue front: first
PriorityQueue	Chèn 4, 9, 1 vào Max-Heap, lấy phần tử có độ ưu tiên cao nhất.	PriorityQueue extract: 9
Algorithms	Sắp xếp trộn mảng {5, 2, 9, 1, 5, 6} và tìm kiếm nhị phân phần tử 5.	Sorted: 1 2 5 5 6 9 Index of 5: 3
BST & AVL	Chèn {8, 3, 10} vào BST và {30, 20, 10} vào AVL. Duyệt inorder và đo chiều cao.	BST inorder: 3 8 10 AVL inorder: 10 20 30 AVL height: 2
HashTable	Chèn 'alice': 90, 'bob': 85, ghi đè 'alice': 95. Kiểm tra tìm kiếm và chứa.	alice: 95 contains bob: 1 (true)

Kết quả chạy thực tế (Chương trình test_algoracon.exe):

```
LinkedList forward: 1 2 3
LinkedList backward: 3 2 1
Stack top: 20
Queue front: first
PriorityQueue extract: 9
Sorted array: 1 2 5 5 6 9
Index of 5: 3
BST inorder: 3 8 10
```

```
AVL inorder: 10 20 30
AVL height: 2
HashTable alice: 95
HashTable contains bob: 1
```

Nhan phim Enter de thoat...

2. Kiểm thử ứng dụng giả lập Cache (app/main.cpp)

Ứng dụng chạy đúng theo kịch bản tương tác dòng lệnh (CLI). Khi khởi chạy, chương trình thực hiện các bước:

- **Khởi tạo:** Khởi tạo thành công bộ mô phỏng Cache kích thước 5 blocks, bộ nhớ chính có 8 blocks chứa dữ liệu từ Data_Block_0 đến Data_Block_7.
- **Yêu cầu:** Cho phép thêm yêu cầu Đọc/Ghi địa chỉ ảo bằng bàn phím (lưu vào hàng đợi Queue).
- **Thực thi:** Khi chọn 'Step' hoặc 'Process All', hệ thống xử lý yêu cầu, hiển thị bảng trạng thái Cache cập nhật theo thời gian thực (được điều hướng tự động bằng giải thuật LRU).
- **Báo cáo:** Xem được báo cáo sắp xếp giảm dần theo lượt truy cập sử dụng Bubble Sort và danh sách khối nhớ xếp theo thứ tự ưu tiên giải phóng LFU thông qua Priority Queue.

Chương trình chạy ổn định, không gặp lỗi rò rỉ bộ nhớ hoặc lỗi logic trong quá trình tương tác CLI.